PlugPlan SG - AI Technology Review and Skeleton

SC2006 | Team SCE3-04 | Lab 3, sections 3.3.1-3.3.5
AI model: GPT-6-luna | 25 October 2026

Inputs: Lab 1 system-description summary, the updated design class diagram and system architecture.
Preferences: TypeScript, Next.js/React and Supabase.

1. Technology-stack recommendation

Prompt

```
You are advising SC2006 Team SCE3-04 on PlugPlan SG. Read these supplied materials:
D:/Downloads/SC2006-SCE3-04/Lab 3/ai-evidence/system-description.md
D:/Downloads/SC2006-SCE3-04/Lab 3/classDiagram.md
D:/Downloads/SC2006-SCE3-04/Lab 3/systemArchitecture.md

Recommend a technology stack appropriate for a small student team. Our preferences are TypeScript, React with
Next.js, and Supabase for authentication, PostgreSQL and private image storage. Explain how your choices support
the design's boundary/control modules, immutable Plan Versions, government-data adapters, role-restricted
moderation and rank-then-filter fault handling. Keep the recommendation-only scope and manual refresh. Identify
any unnecessary complexity and suggest a simpler approach where appropriate.

Write a concise original response in English, about 200-250 words. Include at least one technology that is not a
programming language. Verify technology capability claims using primary official documentation and include two or
three source URLs. Save your complete response verbatim to D:/Downloads/SC2006-SCE3-04/Lab 3/ai-evidence/stack-
response.md and return the identical text. Do not edit the supplied inputs or write the team's evaluation.
```

AI response

Use one TypeScript codebase: React screens in the Next.js App Router, with Route Handlers as the control boundary. They provide custom Web Request/Response handlers for authentication, validation, dispatch, and error mapping (Next.js docs). Keep plan calculation pure. Thin repositories and two server-side adapters normalize LTA DataMall and OneMap responses, apply deadlines, and label live, cached, or stale data. Use Leaflet for the map and OneMap tiles.

Use Supabase Auth, PostgreSQL, and private Storage. Forward the signed-in user’s JWT to database calls; enforce driver ownership with RLS, and gate moderator actions in the server and database. Supabase RLS policies provide database-side authorization using Auth context; private buckets apply RLS to objects (RLS; private buckets). Store moderator roles in trusted, server-controlled data and audit each moderation action with actor, reason, and revision.

Store immutable PlanVersions as JSONB rows, using the design’s update-rejecting SQL trigger and same-plan version constraint. For creation, reconfiguration, and manual refresh, rank feasible stations first, then filter only moderator-verified, unexpired faults. Preserve evidence and explain a short or empty list; pending reports never exclude a station.

Skip microservices, queues, an ORM, a separate auth service, and automatic refresh. One Next.js deployment and Supabase are sufficient for a student team. Main risk is provider changes or outages; keep adapters isolated and label manual refresh as a new version.

2. Comparison with Lab 2

Follow-up prompt

```
Please shorten the final sentence of stack-response.md so it ends after the practical recommendation, without a general disclaimer. Then compare your recommended TypeScript/Next.js/Supabase stack with the previous Lab 2 AI suggestion of Python FastAPI, PostgreSQL and a thin browser UI. Lab 2 did not specify a preferred programming language. Explain briefly what stays the same, what changes, and whether the current preferences fit the updated design. Use about 100 words in English. Save the comparison verbatim to D:/Downloads/SC2006-SCE3-04/Lab 3/ai-evidence/comparison-response.md and return it. Do not write the team's evaluation.
```

AI response

The architecture stays a modular monolith: PostgreSQL constraints and transactions, immutable Plan Versions, government-data adapters, server-side moderator authorization, rank-then-filter fault handling, and manual refresh. The stack changes from Lab 2's Python FastAPI plus thin browser UI to TypeScript, React, and Next.js Route Handlers in one UI/API codebase; Supabase adds managed Auth and private image storage around PostgreSQL. Lab 2 did not specify a language preference, so FastAPI was provisional rather than a requirement. The current preferences fit the updated boundary/control design: Route Handlers coordinate use cases, while Supabase RLS reinforces ownership and role restrictions. Keep calculation and adapters separate; queues and automatic refresh remain unnecessary.

Team evaluation - 3.3.2

The recommendations are mostly the same architecturally: both use a modular monolith, PostgreSQL, provider adapters and immutable Plan Versions. The language/framework changes to TypeScript and Next.js, with Supabase handling Auth and private Storage. This fits our stated preferences and the updated diagram's browser, control and repository boundaries, so we agree with the revised choice.

3. Minimal skeleton generation

Prompt

```
Using the supplied updated class diagram and the selected TypeScript/Next.js/Supabase stack, generate a minimal project folder hierarchy for PlugPlan SG. Map React boundaries to src/components, shared entities and value types to src/lib/types.ts, API/control modules to src/app/api and src/lib/server, pure calculations to src/lib/domain, and database setup to supabase/migrations.

Every file must be empty or contain TODO comments only: no imports, declarations, implementation code, dependency lists, SQL statements, credentials or sample values. Keep JSON files empty because JSON does not support comments. Use native comment syntax in other file types. Include the app entry/layout/styles, API and health routes, client helper, map and operator-support helpers, domain modules, server modules, configuration placeholders, README, an environment example and one migration placeholder. Do not create tests, build artifacts or dependencies.

Write only to D:/Downloads/SC2006-SCE3-04/Lab 3/ai-generated-skeleton/ and package that folder as D:/Downloads/SC2006-SCE3-04/Lab 3/AI-Generated-Skeleton.zip. Keep it separate from application skeleton. Save a concise English textual response including the complete folder tree to D:/Downloads/SC2006-SCE3-04/Lab 3/ai-evidence/skeleton-response.md and return the identical text. Do not edit any supplied input or the full application.
```

AI response

Created the standalone skeleton at `D:\Downloads\SC2006-SCE3-04\Lab 3\ai-generated-skeleton\` and packaged it as `D:\Downloads\SC2006-SCE3-04\Lab 3\AI-Generated-Skeleton.zip`.

```
ai-generated-skeleton/
├── .env.example
├── .gitignore
├── README.md
├── eslint.config.mjs
├── next-env.d.ts
├── next.config.ts
├── package.json
├── postcss.config.mjs
├── tsconfig.json
├── src/
│   ├── app/
│   │   ├── api/
│   │   │   ├── app/
│   │   │   │   └── route.ts
│   │   │   └── health/
│   │   │       └── route.ts
│   │   ├── globals.css
│   │   ├── layout.tsx
│   │   └── page.tsx
│   ├── components/
│   │   ├── auth-screen.tsx
│   │   ├── button.tsx
│   │   ├── data-badge.tsx
│   │   ├── empty.tsx
│   │   ├── field.tsx
│   │   ├── journey-map.tsx
│   │   ├── modal.tsx
│   │   ├── place-picker.tsx
│   │   ├── plugplan-app.tsx
│   │   └── status-badge.tsx
│   └── lib/
│       ├── client.ts
│       ├── types.ts
│       ├── domain/
│       │   ├── engine.ts
│       │   ├── fixtures.ts
│       │   └── validation.ts
│       ├── map/
│       │   └── polyline.ts
│       └── server/
│           ├── auth.ts
│           ├── db.ts
│           ├── errors.ts
│           ├── operator-support.ts
│           ├── provider-cache.ts
│           ├── providers.ts
│           └── service.ts
└── supabase/
    └── migrations/
        └── 0001_placeholder.sql
```

All non-JSON files contain TODO comments only; `package.json` and `tsconfig.json` are empty. The skeleton contains no tests, build artifacts, or dependencies.

Team evaluation - 3.3.4

The folders cover the main UI, control, domain, provider and persistence responsibilities. Some UI helpers are split more finely than necessary; the PostCSS and ESLint placeholders can also be omitted from our plain-CSS setup. No implementation, imports, declarations or SQL statements were added: all 38 files contain TODO comments only or are empty JSON files. The dependency configuration and migration bodies remain to be filled as part of developing the skeleton.

Submitted output: `AI-Generated-Skeleton.zip`. The complete application is provided separately in `application skeleton/`.